



UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Analisi dello Stato dell'Arte

Tecnologie e Soluzioni per la Realizzazione del Sistema

Gruppo: Synergo Unit

Email: synergounit.20@gmail.com

Versione: **v0.0.0**
Data: **2026-05-18**

Registro delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
0.0.0	2026-05-18	Francesco Marcon	Anton R. Rupi	Prima stesura del documento.

Indice

1	Introduzione	3
1.1	Obbiettivi del Documento	3
1.2	Aree di Indagine	3
2	Tecnologie Frontend	3
3	Tecnologie Backend	4
4	Architettura	5
5	Sicurezza e Protezione dei Dati	6
6	Scelte Finali	6

1 Introduzione

1.1 Obbiettivi del Documento

Il presente documento raccoglie i risultati della ricerca preliminare (“Stato dell’Arte”) sulle tecnologie e tecniche che il gruppo SynergoUnit utilizzerà per portare a termine lo sviluppo del PoC, atto a dimostrare la fattibilità del sistema richiesto dalla proponente Zucchetti, presentato nel capitolato C6 - Second Brain, garantendo una soluzione moderna, scalabile e manutenibile.

1.2 Aree di Indagine

L’analisi verte sulle seguenti aree di sviluppo.

- Tecnologie frontend: valutazione di framework e librerie per interfacce reattive.
- Tecnologie backend: analisi di framework per API RESTful e gestione logica.
- Architettura: definizione della struttura del sistema e gestione del deployment.
- Sicurezza e protezione dei dati: strategie per la protezione delle comunicazioni e dei dati sensibili.

2 Tecnologie Frontend

Il gruppo ha condotto un’analisi comparativa sui principali framework e librerie JavaScript / TypeScript attualmente leader del settore. L’obiettivo era individuare lo strumento più idoneo per lo sviluppo di un’interfaccia reattiva, capace di gestire la necessità di mantenere il sistema semplice e intuitivo anche per utenti non esperti.

Sono state analizzate le seguenti alternative.

- **Angular:** framework vastamente utilizzato e completo.
 - *Pro:* offre una struttura robusta e strumenti integrati (routing, gestione stati, client HTTP) senza dipendere da librerie terze.
 - *Contro:* curva di apprendimento estremamente ripida, è stato ritenuto eccessivamente pesante per gli obiettivi del sistema richiesto da Zucchetti.
- **React:** principale alternativa ad Angular per diffusione e integrazione.
 - *Pro:* ecosistema maturo, performance ottimizzate, ampia disponibilità di librerie e pattern consolidati.
 - *Contro:* richiede la definizione preventiva di standard architetturali (state management, routing, styling) per evitare frammentazione nel team. Data l’assenza di esperienza pregressa con framework moderni, è stato valutato come ad alto rischio di debito tecnico iniziale.
- **Vue.js:** tecnologia scelta dal gruppo per lo sviluppo frontend.
 - *Pro:* progressività (inizio da codice semplice e intuitivo, man mano si aggiunge complessità ove necessario), separazione di struttura HTML e styling CSS, ottimizzato per granularità della User Interface.

- *Contro*: ecosistema leggermente meno vasto di React in ambito enterprise, ma pienamente sufficiente per le funzionalità richieste dal capitolato.

Inoltre, nelle tecnologie frontend ricade la scelta di un altro componente critico: l'editor di testo. Le alternative analizzate in questo caso sono state due.

- **Monaco Editor**: motore alla base dell'editor di testo VS Code, potente e completo ma con elevato consumo di risorse ed una complessa configurazione.
- **Code Mirror**: componente scelto dal gruppo per bilanciamento tra leggerezza e estensibilità. Ideale per l'implementazione del sistema Second Brain, fornisce la possibilità di integrare comandi personalizzati per l'interazione con il LLM e/o altre feature del sistema, ha un ecosistema di estensioni pronte all'uso per Markdown, syntax highlighting, keybindings, ...

Sono stati presi in considerazione anche altri editor, come TipTap e Milkdown, tuttavia sono stati scartati per la complessità di configurazione, inoltre richiedono maggiore esperienza per mantenere la coerenza tra modello interno e markdown generato.

3 Tecnologie Backend

La scelta dell'ecosistema backend è stata guidata da tre requisiti critici: la necessità di elaborare dati testuali tramite modelli di intelligenza artificiale (LLM), la velocità di risposta delle API e la facilità di integrazione con il database relazionale.

Come linguaggio di programmazione, per conoscenza ed esperienza pregressa di alcuni membri del gruppo, valutando anche la complessità ridotta della curva di apprendimento, è stato scelto direttamente **Python**.

Per quanto riguarda, invece, i framework di gestione delle api, sono state valutate le seguenti alternative.

- **Django**: soluzione API più completa per python.
 - *Pro*: soluzione pronta all'uso su tutti i fronti, sicurezza integrata di alto livello.
 - *Contro*: eccessivamente pesante e rigido per un sistema semplice e dove la rapidità è chiave, la sua natura sincrona lo rende meno adatto a lavori di parallelizzazione.
- **Flask**: estremamente leggero.
 - *Pro*: semplice da configurare, ideale per piccoli prototipi.
 - *Contro*: manca di strumenti all'avanguardia, validazione dei dati o supporto asincrono, richiede l'uso di "plugin" esterni per raggiungere le funzionalità minime richieste.
- **Fast API**: tecnologia scelta dal gruppo.
 - *Pro*: elevate prestazioni, pensato per chiamate asincrone (fondamentali nelle chiamate al LLM), supporta la validazione dei dati, genera la documentazione in modo automatico utilizzando Swagger, permette facile gestione di middlewares per logging e sicurezza, integrazione di rate limiting (protezione delle API dei LLM).
 - *Contro*: curva di apprendimento leggermente più complessa rispetto ad alternative basilari come Flask.

La gestione dei dati (salvataggio di note), pur essendo un requisito opzionale, richiede la valutazione di un sistema di database.

Durante l'analisi, il gruppo ha analizzato sia un approccio SQL, sia un approccio NoSQL. Rispettivamente, i motori selezionati sarebbero stati PostgreSQL (preferito dalla proponente Zucchetti) e MongoDB (flessibile nel gestire documenti non strutturati).

La scelta finale è però ricaduta su **PostgreSQL**.

- Sistema relazionale più solido rispetto ad un sistema NoSQL.
- Affidabile gestione delle transazioni.
- Indexing avanzato per ricerche full-text.
- Licenza open source, a differenza della licenza più restrittiva di MongoDB.
- Supporto al salvataggio di informazioni JSON, offrendo la flessibilità dei database non relazionali pur mantenendo la robustezza di un sistema SQL.
- Supporto al salvataggio di embeddings (tramite plugin), permette infatti di salvare dati che possono essere dati direttamente in pasto ai LLMs.

4 Architettura

L'architettura software può essere progettata seguendo due approcci principali, ciascuno con caratteristiche, vantaggi e compromessi differenti.

1. **Architettura Monolitica:** il sistema è sviluppato come un'unica applicazione integrata, nella quale frontend, backend, accesso ai dati e gestione della sicurezza sono strettamente collegati all'interno di un singolo progetto o pacchetto distribuibile. Questo approccio semplifica lo sviluppo iniziale e il deployment, ma può risultare meno flessibile nella scalabilità e nella manutenzione di sistemi complessi.
2. **Architettura a Microservizi:** il sistema è suddiviso in servizi indipendenti e specializzati, ciascuno responsabile di una specifica funzionalità. Ogni microservizio può essere sviluppato, distribuito e scalato autonomamente, migliorando modularità, resilienza ai guasti e manutenibilità, a fronte di una maggiore complessità nella gestione dell'infrastruttura e della comunicazione tra servizi.

La scelta del gruppo è ricaduta su un'architettura **monolitica modulare**, dove saranno definiti 3 *moduli*:

- Frontend
- Backend
- Database

Per lo strumento di deploy dei 3 moduli, il gruppo ha scelto **Docker**: una piattaforma di containerizzazione che permette di isolare e distribuire le componenti software in ambienti leggeri e riproducibili.

Ogni container è considerato *lightweight* poiché, a differenza delle tradizionali macchine virtuali, non richiede l'emulazione completa di un sistema operativo. Esso si basa invece su un'immagine

essenziale, contenente esclusivamente le dipendenze e i file necessari al funzionamento dello specifico servizio.

Ad esempio, un container basato su PostgreSQL include solamente i componenti indispensabili per l'esecuzione del database, garantendo così maggiore efficienza, rapidità di avvio e portabilità tra ambienti differenti.

Nel nostro caso, Docker viene utilizzato per containerizzare le 3 componenti definite prima, per renderle leggere, portabili, e facilmente avviabili, e favorire così anche lo sviluppo del software da parte dei membri del gruppo.

5 Sicurezza e Protezione dei Dati

La natura del sistema *Second Brain* implica l'archiviazione di informazioni personali, note private e potenziale proprietà intellettuale, oltre a una costante comunicazione con modelli di intelligenza artificiale esterni.

La strategia di sicurezza del gruppo si concentra sulla protezione dei dati sia in transito sia a riposo, strutturandosi sui seguenti pilastri.

- **Cifratura dei dati in transito (Data in Transit):** tutte le comunicazioni tra il frontend (Vue.js) e il backend (FastAPI) saranno protette tramite protocollo HTTPS con cifratura TLS. Questo approccio previene attacchi di tipo Man-in-the-Middle (MitM) e intercettazioni di testo sensibile.
- **Autenticazione e Autorizzazione:** l'accesso al sistema e alle relative API sarà regolato da un meccanismo di autenticazione basato su token (es. JWT - JSON Web Token), implementato nativamente tramite i middleware di FastAPI. In questo modo si garantisce l'isolamento dei dati, assicurando che ciascun utente possa visualizzare e modificare solo le proprie note e il proprio grafo delle relazioni.
- **Gestione Sicura delle Credenziali:** le API key per l'accesso agli LLM e le stringhe di connessione al database PostgreSQL non saranno mai inserite direttamente nel codice sorgente (hardcoded). Saranno invece gestite come variabili d'ambiente protette all'interno dell'ambiente di containerizzazione Docker, riducendo drasticamente il rischio di leak accidentali.
- **Privacy dei dati e conformità con gli LLM:** poiché il sistema invia porzioni di testo a modelli esterni per l'applicazione di filtri (come il *Distant Writing* o l'analisi critica dei *Sei Cappelli per Pensare*), il backend implementerà una logica di sanificazione preliminare per intercettare ed evitare l'invio involontario di dati personali identificativi (PII). Inoltre, l'adozione del *rate limiting* su FastAPI proteggerà l'infrastruttura da abusi e picchi di costo incontrollati derivanti dalle chiamate ai modelli.

6 Scelte Finali

L'analisi approfondita dello Stato dell'Arte ha permesso al gruppo SynergoUnit di definire uno stack tecnologico solido, moderno e strettamente mirato a soddisfare i requisiti del capitolato C6 proposto da Zucchetti.

La sinergia tra le tecnologie selezionate definisce la seguente architettura di riferimento per il PoC.

- **Frontend (Vue.js + CodeMirror)**: offre un'interfaccia utente (UI) reattiva, leggera e caratterizzata da una bassa curva di apprendimento. L'integrazione di CodeMirror garantisce la flessibilità necessaria per manipolare il testo in formato Markdown, permettendo lo sviluppo di plug-in personalizzati per comunicare con l'LLM direttamente dalla finestra di scrittura.
- **Backend (Python + FastAPI)**: fornisce un ecosistema ad alte prestazioni e nativamente asincrono. Questa caratteristica è fondamentale per gestire i tempi di latenza tipici delle risposte degli LLM (per riscritture, riassunti o traduzioni) senza bloccare l'esperienza d'uso dell'utente.
- **Database (PostgreSQL)**: pur trattandosi di un requisito opzionale, la scelta di PostgreSQL unisce l'affidabilità di un sistema relazionale SQL alla flessibilità della gestione di dati non strutturati (tramite JSON). Inoltre, il supporto nativo agli embeddings tramite estensioni permette di mappare in modo efficiente il grafo delle relazioni tra le note ricavato dall'Intelligenza Artificiale.
- **Deployment (Docker)**: l'approccio a microservizi tramite container garantisce la massima portabilità e isolamento dei componenti, azzerando i conflitti di configurazione locale tra i membri del team e ponendo le basi per una futura scalabilità industriale.

In conclusione, il gruppo ritiene che questo stack rappresenti il compromesso ideale tra velocità di sviluppo e robustezza architetturale.

La soluzione così delineata permetterà di validare l'efficacia del sistema di **Second Brain**, traghettando il note-taking tradizionale verso il nuovo paradigma del co-authoring e del distant writing descritto nella proposta della proponente.